

Modelo MILP de Keccak reducido con función χ modificada

1. Introducción

Keccak es una función esponja utilizada como base del estándar criptográfico SHA-3. Su función de permutación interna está formada por cinco transformaciones principales:

$$\theta, \rho, \pi, \chi, \iota$$

En este trabajo se implementa un modelo de Programación Lineal Entera Mixta (MILP) para analizar una versión reducida de Keccak.

El objetivo del modelo es encontrar el número mínimo de cajas χ activas necesarias para una ronda reducida, utilizando un enfoque de propagación diferencial.

El problema de optimización es:

$$\min \sum ChiActive$$

donde:

- $ChiActive$ representa si una caja χ presenta actividad diferencial.
- La minimización busca encontrar la menor cantidad posible de cajas χ activas.

La configuración utilizada es:

Parámetro	Valor
Número de rondas	1
Dimensión del estado	5 × 5 lanes
Bits por lane	4
Tamaño total del estado	100 bits
Solver	Gurobi 13.0.2

2. Representación del estado Keccak reducido

El estado interno de Keccak se representa como:

$$S[x, y, z]$$

donde:

- x representa la coordenada horizontal del lane.
- y representa la coordenada vertical.

- z representa la posición del bit dentro del lane.

Para el caso reducido:

$$x, y \in \{0, \dots, 4\}$$

y:

$$z \in \{0, \dots, 3\}$$

Por lo tanto, el número total de bits es:

$$5 \times 5 \times 4 = 100$$

Cada variable:

$$S[x, y, z] \in \{0, 1\}$$

representa la presencia o ausencia de actividad diferencial.

3. Transformación θ

La transformación θ calcula la paridad de cada columna del estado.

Primero se define:

$$C[x, z] = S[x, 0, z] \oplus S[x, 1, z] \oplus S[x, 2, z] \oplus S[x, 3, z] \oplus S[x, 4, z]$$

donde $C[x, z]$ representa la paridad de una columna.

Para reducir la cantidad de variables auxiliares, la paridad se modela utilizando:

$$\sum_{y=0}^4 S[x, y, z] = C[x, z] + 2k$$

donde:

- $C[x, z]$ es una variable binaria.
- k es una variable entera auxiliar.

Luego se calcula:

$$D[x, z] = C[x - 1, z] \oplus C[x + 1, z - 1]$$

La salida de θ es:

$$B[x, y, z] = S[x, y, z] \oplus D[x, z]$$

4. Modelado MILP de la operación XOR

Las operaciones XOR no lineales se representan mediante restricciones lineales binarias.

Para:

$$c = a \oplus b$$

la formulación MILP utilizada es:

$$c \leq a + b$$

$$c \geq a - b$$

$$c \geq b - a$$

$$c \leq 2 - a - b$$

Estas restricciones garantizan que:

a	b	c
0	0	0
0	1	1
1	0	1
1	1	0

Esta representación evita crear tablas de verdad completas y permite utilizar un solver MILP.

5. Transformaciones ρ y π

Después de θ se aplican las transformaciones:

$$\rho + \pi$$

La transformación ρ realiza una rotación dentro de cada lane.

Si $r[x, y]$ representa el desplazamiento de rotación:

$$B'[x, y, z] = B[x, y, (z - r[x, y]) \bmod 4]$$

La transformación π reorganiza las posiciones:

$$(x, y) \rightarrow (y, 2x + 3y)$$

En el modelo MILP, la salida de estas transformaciones se representa mediante:

$$ChiInput[x, y, z]$$

6. Función χ original de Keccak

La transformación χ original es la única operación no lineal de Keccak.

Está definida como:

$$A'_x = A_x \oplus ((\neg A_{x+1}) \wedge A_{x+2})$$

Cada salida depende de tres entradas:

$$A_x, A_{x+1}, A_{x+2}$$

La parte AND se representa mediante una variable auxiliar:

$$ChiAND$$

con las restricciones:

$$ChiAND \leq A_{x+2}$$

$$ChiAND \leq 1 - A_{x+1}$$

$$ChiAND \geq A_{x+2} - A_{x+1}$$

Posteriormente:

$$T = A_x \oplus ChiAND$$

y:

$$A'_x = T$$

7. Modificación propuesta de χ

Para incrementar la dependencia entre bits dentro de la caja χ se propone agregar un XOR adicional:

$$A'_x = A_x \oplus ((\neg A_{x+1}) \wedge A_{x+2}) \oplus A_{x+3}$$

La diferencia respecto a Keccak original es:

$$\oplus A_{x+3}$$

Por lo tanto, la caja modificada utiliza cuatro entradas:

$$A_x, A_{x+1}, A_{x+2}, A_{x+3}$$

La implementación MILP realiza:

Primero:

$$T = A_x \oplus ((\neg A_{x+1}) \wedge A_{x+2})$$

Después:

$$A'_x = T \oplus A_{x+3}$$

La segunda operación XOR requiere restricciones adicionales en el modelo MILP.

8. Variable de actividad χ

Cada caja χ procesa cinco bits:

$$x = 0, \dots, 4$$

La actividad de una caja se define como:

$$ChiActive[y, z]$$

donde:

$$ChiActive[y, z] = 1$$

si al menos uno de los cinco bits de entrada tiene actividad.

Se utilizan las restricciones:

$$ChiActive[y, z] \geq ChiInput[x, y, z]$$

para todo:

$$x = 0, \dots, 4$$

y:

$$ChiActive[y, z] \leq \sum_{x=0}^4 ChiInput[x, y, z]$$

9. Función objetivo MILP

El objetivo es minimizar el número total de cajas χ activas:

$$\min \sum_r \sum_y \sum_z ChiActive[r, y, z]$$

Además se imponen condiciones para evitar la solución trivial:

Estado inicial:

$$\sum S[0, x, y, z] \geq 1$$

Estado final:

$$\sum S[rounds, x, y, z] \geq 1$$

Estas restricciones obligan a que exista actividad diferencial.

10. Resultados experimentales

Se evaluaron dos modelos:

- 1. Keccak reducido con χ original.
- 2. Keccak reducido con χ modificada.

10.1 χ original

Resultados obtenidos con Gurobi:

Parámetro	Resultado
Variables	680
Restricciones	1522
Coeficientes no nulos	4400
Tiempo	0.5352 s
Estado	OPTIMAL
Mínimo χ activo	1

Resultado:

$$\min \sum ChiActive = 1$$

10.2 χ modificada

Resultados obtenidos con Gurobi:

Parámetro	Resultado
Variables	680
Restricciones	1822
Coeficientes no nulos	5400
Tiempo	0.1400 s
Estado	OPTIMAL
Mínimo χ activo	1

Resultado:

$$\min \sum ChiActive = 1$$

11. Comparación de resultados

Característica	χ original	χ modificada
Función	$A_x \oplus ((\neg A_{x+1}) \wedge A_{x+2})$	$A_x \oplus ((\neg A_{x+1}) \wedge A_{x+2}) \oplus A_{x+3}$
Bits involucrados	3	4
Variables MILP	680	680
Restricciones	1522	1822
Coefficientes no nulos	4400	5400
Tiempo ejecución	0.5352 s	0.1400 s
χ activo mínimo	1	1

12. Análisis

Los dos modelos alcanzan la misma solución óptima:

$$\min \sum ChiActive = 1$$

Esto indica que para una sola ronda reducida y palabras de 4 bits, la modificación propuesta no incrementa la actividad diferencial mínima.

La modificación de χ aumenta la dependencia local porque cada salida depende de un bit adicional:

Original:

$$(A_x, A_{x+1}, A_{x+2})$$

Modificada:

$$(A_x, A_{x+1}, A_{x+2}, A_{x+3})$$

Sin embargo, esta mayor dependencia no se refleja en un aumento del número mínimo de cajas activas.

Desde el punto de vista MILP, la modificación genera:

- Igual número de variables.
- Mayor número de restricciones.
- Mayor cantidad de coeficientes en la matriz del modelo.

Esto se debe a la operación XOR adicional:

$$T \oplus A_{x+3}$$

que requiere una nueva relación lineal binaria.

El menor tiempo observado en la versión modificada no representa necesariamente una menor complejidad criptográfica, ya que depende del proceso interno de búsqueda del solver Gurobi.

Para evaluar completamente el impacto de la modificación se requieren experimentos con:

- más rondas,
- palabras de mayor tamaño,
- estados menos reducidos.

Resultado de ejecución original:

RESOLVIENDO MODELO MILP

Variables: 680 Restricciones: 1522 Gurobi Optimizer version 13.0.2 build v13.0.2rc1 (win64 - Windows 11+.0 (26200.2))

CPU model: Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz, instruction set [SSE2|AVX|AVX2] Thread count: 4 physical cores, 8 logical processors, using up to 8 threads

Non-default parameters: TimeLimit 300

Optimize a model with 1522 rows, 680 columns and 4400 nonzeros (Min) Model fingerprint: 0xd87ee3db
Model has 20 linear objective coefficients Variable types: 0 continuous, 680 integer (660 binary) Coefficient statistics: Matrix range [1e+00, 2e+00] Objective range [1e+00, 1e+00] Bounds range [1e+00, 5e+00] RHS range [1e+00, 2e+00]

Presolve removed 620 rows and 300 columns Presolve time: 0.02s Presolved: 902 rows, 380 columns, 2980 nonzeros Variable types: 0 continuous, 380 integer (360 binary) Found heuristic solution: objective 20.0000000 Found heuristic solution: objective 19.0000000 Found heuristic solution: objective 9.0000000

Root relaxation: objective 1.000000e-01, 428 iterations, 0.01 seconds (0.01 work units)

Nodes		Current Node		Objective Bounds		Work	
-------	--	--------------	--	------------------	--	------	--

Expl Unexpl | Obj Depth IntlInf | Incumbent BestBd Gap | It/Node Time

0	0	0.10000	0	58	9.00000	0.10000	98.9%	-	0s
---	---	---------	---	----	---------	---------	-------	---	----

H 0 0 8.0000000 0.10000 98.8% - 0s H 0 0 2.0000000 0.10000 95.0% - 0s 0 0 0.10000 0 76 2.00000 0.10000 95.0% - 0s 0 0 1.00000 0 80 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 80 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 69 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 82 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 89 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 91 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 122 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 122 2.00000 1.00000 50.0% - 0s 0 0 1.00000 0 37 2.00000 1.00000 50.0% - 0s 0 2 1.00000 0 32 2.00000 1.00000 50.0% - 0s

- 83 49 24 1.00000000 1.000000 0.00% 57.3 0s

Cutting planes: Gomory: 3 Cover: 2 Clique: 22 MIR: 8 StrongCG: 1 Zero half: 1 RLT: 15 BQP: 2

Explored 99 nodes (7812 simplex iterations) in 0.50 seconds (0.22 work units) Thread count was 8 (of 8 available processors)

Solution count 6: 1 2 8 ... 20

Optimal solution found (tolerance 1.00e-04) Best objective 1.00000000000000e+00, best bound 1.00000000000000e+00, gap 0.0000%

RESULTADOS

Estado OPTIMAL Mínimo Chi activo: 1.0 Tiempo: 0.535205602645874 Ronda 0 : 1

Resultado de Ejecución Modificado:

RESOLVIENDO MODELO MILP

Variables: 680 Restricciones: 1822 Gurobi Optimizer version 13.0.2 build v13.0.2rc1 (win64 - Windows 11+.0 (26200.2))

CPU model: Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz, instruction set [SSE2|AVX|AVX2] Thread count: 4 physical cores, 8 logical processors, using up to 8 threads

Non-default parameters: TimeLimit 300

Optimize a model with 1822 rows, 680 columns and 5400 nonzeros (Min) Model fingerprint: 0x57a8089e Model has 20 linear objective coefficients Variable types: 0 continuous, 680 integer (660 binary) Coefficient statistics: Matrix range [1e+00, 2e+00] Objective range [1e+00, 1e+00] Bounds range [1e+00, 5e+00] RHS range [1e+00, 2e+00]

Presolve removed 620 rows and 300 columns Presolve time: 0.03s Presolved: 1202 rows, 380 columns, 4580 nonzeros Variable types: 0 continuous, 380 integer (360 binary) Found heuristic solution: objective 20.0000000 Found heuristic solution: objective 9.0000000

Root relaxation: objective 6.666667e-02, 491 iterations, 0.01 seconds (0.02 work units)

Nodes	Current Node	Objective Bounds	Work
-------	--------------	------------------	------

Expl Unexpl | Obj Depth Intlntf | Incumbent BestBd Gap | It/Node Time

0	0	0.06667	0	31	9.00000	0.06667	99.3%	-	0s
---	---	---------	---	----	---------	---------	-------	---	----

H 0 0 7.0000000 0.06667 99.0% - 0s H 0 0 1.0000000 0.06667 93.3% - 0s

Cutting planes: Gomory: 5 MIR: 1 Zero half: 1 RLT: 4

Explored 1 nodes (673 simplex iterations) in 0.12 seconds (0.05 work units) Thread count was 8 (of 8 available processors)

Solution count 4: 1 7 9 20

Optimal solution found (tolerance 1.00e-04) Best objective 1.00000000000000e+00, best bound 1.00000000000000e+00, gap 0.0000%

RESULTADOS

Estado OPTIMAL Mínimo Chi activo: 1.0 Tiempo: 0.14002013206481934 Ronda 0 : 1

13. Conclusiones

Se implementó un modelo MILP reducido de Keccak y una variante modificada de la función χ .

La modificación consiste en:

$$A'_x = A_x \oplus ((\neg A_{x+1}) \wedge A_{x+2}) \oplus A_{x+3}$$

Los resultados muestran que ambas configuraciones presentan:

$$\min \sum ChiActive = 1$$

Por lo tanto, para la configuración evaluada, agregar el XOR adicional no cambia la actividad diferencial mínima.

No obstante, la modificación aumenta la representación MILP al requerir más restricciones para modelar la nueva dependencia.

Estos resultados muestran que una modificación algebraica de χ puede aumentar la dependencia entre bits sin necesariamente modificar las propiedades diferenciales observadas en una ronda reducida.

Un análisis más profundo requiere extender el modelo a configuraciones más cercanas al comportamiento real de Keccak.